sql刷题笔记

2025年12月22日 18:40

sql学习笔记

分类	类型	大小	描述
字符串类型	CHAR	0-255 bytes	定长字符串
	VARCHAR	0-65535 bytes	变长字符串
	TINYBLOB	0-255 bytes	不超过255个字符的二进制数据
	TINYTEXT	0-255 bytes	短文本字符串
	BLOB	0-65 535 bytes	二进制形式的长文本数据
	TEXT	0-65 535 bytes	长文本数据
	MEDIUMBLOB	0-16 777 215 bytes	二进制形式的中等长度文本数据
	MEDIUMTEXT	0-16 777 215 bytes	中等长度文本数据
	LONGBLOB	0-4 294 967 295 bytes	二进制形式的极大文本数据
	LONGTEXT	0-4 294 967 295 bytes	极大文本数据

char(10) ----------> 性能好

varchar(10) ---------> 性能较差

用户名 username varchar(50)

性别 gender char(1)

分类	类型	大小	范围	格式	描述
日期类型	DATE	3	1000-01-01 至 9999-12-31	YYYY-MM-DD	日期值
	TIME	3	-838:59:59 至 838:59:59	HH:MM:SS	时间值或持续时间
	YEAR	1	1901 至 2155	YYYY	年份值
	DATETIME	8	1000-01-01 00:00:00 至 9999-12-31 23:59:59	YYYY-MM-DD HH:MM:SS	混合日期和时间值
	TIMESTAMP	4	1970-01-01 00:00:01 至 2038-01-19 03:14:07	YYYY-MM-DD HH:MM:SS	混合日期和时间值，时间戳

birthday]date

多选题 SQL

5. 假设有一个表有联合索引a,b,c。那么在sql查找中，下列哪些组合在where条件下是索引有效?

A a

B b

C a,c

D b,c

正确答案：AC 你的答案：C

官方解析：暂无官方题目解析，去讨论区看看吧！

知识点：SQL

联合索引遵循**最左前缀原则**，即查询从索引的最左边列开始且连续使用索引列时索引有效。表有联合索引 a,b,c，A 选项 a 是最左列，索引有效；C 选项 a,c 中 a 是最左列且 c 在联合索引中，也是连续使用索引列，索引有效；B 选项 b 不是最左列，D 选项 b,c 起始不是最左列，索引均无效。

4. 关于MVCC(多版本并发控制)的实现机制，正确的是：

A 通过行级锁避免读写冲突

B 使用UNDO日志构建数据的历史版本

C 要求所有事务使用串行化隔离级别

D 每次更新数据时都会覆盖原数据页

正确答案：B 你的答案：未作答

官方解析：
选项B正确：在MVCC（多版本并发控制）的实现中，UNDO日志用于存储数据的历史版本，从而允许事务读取一致的历史快照而不阻塞写操作。选项A错误：MVCC的核心是避免使用锁（尤其是读写锁），而是通过多版本控制减少冲突。选项C错误：MVCC支持多种隔离级别（如读已提交、可重复读），并不要求所有事务使用串行化隔离级别。选项D错误：MVCC在更新数据时不会直接覆盖原数据页，而是创建新版本，并保留旧版本供并发读取。

知识点：SQL

db_access admin	可以添加、删除用户的用户

db_datare ader	可以查看所有数据库中用户表内数据的用户

这里主要介绍几个固定数据库角色

角色	说明
db_accessadmin	为windows登录名、windows组和sqlserver登录名添加或删除数据库访问权限
db_backupoperator	备份数据库
db_datareader	从所有用户表中读取所有数据权限
db_datawriter	在所有用户表中添加、删除或更改数据
db_ddladmin	在数据库中运行任何DDL命令权限
db_denydatareader	不能读取数据库内用户表中的任何数据
db_denydatawriter	不能添加、修改或删除数据库内用户表中的任何数据
db_owner	可以执行数据库所有配置和维护活动，还可以删除数据库
db_securityadmin	修改角色成员身份和管理权限，向此角色中添加主体可能或导致意外的权限升级
public	在SQL Server 2008中每个数据库用户都属于public数据库角色。当尚未对某个用户授予或者拒绝对安全对象的特定权限时，这该用户将据称授予该安全对象的public角色的权限。这个数据库角色不能被删除

头条 @波波说运维

2. 在分析电商订单数据时，某开发者需要统计每个用户的总订单金额，但仅包括状态为‘已支付’的订单。以下哪种SQL查询最有效地实现该需求？

A SELECT user_id, SUM(amount) FROM orders WHERE status = '已支付' GROUP BY user_id

B SELECT user_id, SUM(amount) FROM orders GROUP BY user_id HAVING status = '已支付'

C SELECT user_id, AVG(amount) FROM orders WHERE status = '已支付' GROUP BY user_id

D SELECT user_id, COUNT(*) FROM orders WHERE status = '已支付' GROUP BY user_id

正确答案：A

官方解析：
题目要求统计每个用户的总订单金额，仅包括状态为‘已支付’的订单。选项A（SELECT user_id, SUM(amount) FROM orders WHERE status = '已支付' GROUP BY user_id）正确：先使用WHERE子句过滤状态为‘已支付’的订单，再按user_id分组，并用SUM(amount)计算总金额。选项B错误：HAVING子句不能用于未聚合的列（status），它只适用于聚合函数后的结果；选项C错误：使用AVG(amount)计算平均金额而非总金额；选项D错误：使用COUNT(*)统计订单数量而非金额总和。题目文字、逻辑和专业名词无误，无规则破坏错误。

知识点：SQL

HAVING 是分组后的筛选条件，不能直接筛选 status

单选题 SQL

1. 使用MySQL，以下语句中表示“查找出表A和表B中id相同的所有数据”的是?

A SELECT A.*,B.* FROM A a RIGHT JOIN B b ON a.id=b.id

B SELECT A.*,B.* FROM A a JOIN B b ON a.id=b.id

C SELECT A.*,B.* FROM A a LEFT JOIN B b ON a.id=b.id

D SELECT A.*,B.* FROM A a JOIN B b IN a.id=b.id

正确答案：B

官方解析：
答案选择B
A选项表示“查找出表B的所有数据，以及表A中id和表B中id相同的数据”
B选项正确
C选项表示“查找出表A的所有数据，以及表B中id和表A中id相同的数据”
D语法错误

知识点：SQL

- **A 选项（RIGHT JOIN）**：右连接，会返回表 B 的所有记录，以及表 A 中与表 B id 匹配的记录，不满足“表 A 和表 B 中 id 相同的所有数据”。
- **B 选项（JOIN）**：在 MySQL 中，`JOIN` 默认等同于 `INNER JOIN`（内连接），只返回两个表中 id 完全匹配的记录，符合题意。
- **C 选项（LEFT JOIN）**：左连接，会返回表 A 的所有记录，以及表 B 中与表 A id 匹配的记录，不满足题意。
- **D 选项**：语法错误，`JOIN` 的连接条件必须使用 `ON` 关键字，而不是 `IN`。

单选题 SQL

2. 用EXPLAIN分析查询时发现‘Using temporary; Using filesort’，这对性能意味着什么?

A 查询需要临时磁盘存储和外部排序

B 数据库正在使用内存缓存加速

C 表示索引被高效利用无性能风险

D 触发了查询优化器的特殊执行模式

正确答案：A　你的答案：B

官方解析：
在MySQL的EXPLAIN输出中，'Using temporary'表示查询需要创建临时表存储中间结果，可能涉及磁盘存储（当临时表超过内存限制）；'Using filesort'表示排序操作无法在内存中完成，需使用文件系统进行外部排序。这通常导致额外磁盘I/O和性能下降。因此，选项A正确描述了性能影响。选项B错误，因为这些标志表示内存不足或非缓存操作（而非加速）；选项C错误，因为它们常表明索引未被高效利用且有性能风险；选项D错误，这不是特殊执行模式而是常见优化结果。题目无文字错误、逻辑矛盾或专业名称错误。

知识点：SQL

sql难题

单选题 SQL

3. MySQL中score_table表结构如下：

```
CREATE TABLE `score_table` (
  `stu_id` varchar(20) DEFAULT NULL,
  `subject_name` varchar(20) DEFAULT NULL,
  `score` int DEFAULT NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 ;
```

表中全部记录如下：

```
+--------+--------------+-------+
| stu_id | subject_name | score |
+--------+--------------+-------+
| 1004   | 语文         |    90 |
| 1004   | 数学         |    82 |
| 1004   | 英语         |    88 |
| 1009   | 语文         |    91 |
| 1009   | 数学         |    92 |
| 1009   | 英语         |    78 |
| 1010   | 语文         |    71 |
| 1010   | 数学         |    82 |
| 1006   | 语文         |    61 |
| 1012   | 数学         |    80 |
| 1012   | 语文         |    80 |
+--------+--------------+-------+
```

希望得到如下结果：按照subject_name分组后按照score升序排序，percent=（当前记录在分区内的序号-1）除以（总行数）。比如stu_id=1012，在数学分区内序号是1，数学分区内共4条记录，所以percent=(1-1)/(4-1)=0%。对于重复值，取重复值第一行的序号，比如stu_id=1004与stu_id=1010的数学score都是82，第一行的序号是2，所以percent=(2-1)/(4-1)=33.33%。

```
+--------+--------------+-------+---------+
| stu_id | subject_name | score | percent |
+--------+--------------+-------+---------+
| 1012   | 数学         |    80 | 0%      |
| 1004   | 数学         |    82 | 33.33% |
| 1010   | 数学         |    82 | 33.33% |
| 1009   | 数学         |    92 | 100%    |
| 1009   | 英语         |    78 | 0%      |
| 1004   | 英语         |    88 | 100%    |
| 1006   | 语文         |    61 | 0%      |
| 1010   | 语文         |    71 | 25%     |
| 1012   | 语文         |    80 | 50%     |
| 1004   | 语文         |    90 | 75%     |
| 1009   | 语文         |    91 | 100%    |
+--------+--------------+-------+---------+
```

以下SQL符合预期的是（）？

A
```
SELECT stu_id , subject_name ,score ,
concat(round( quantile_rank() over(partition by subject_name order by score),4)*100,'%') AS percent
FROM score_table;
```

B
```
SELECT stu_id , subject_name ,score ,
concat(round( percent_rank() over(partition by subject_name order by score),4)*100,'%') AS percent
FROM score_table;
```

C
```
SELECT stu_id , subject_name ,score ,
concat(round( percentile_rank() over(partition by subject_name order by score),4)*100,'%') AS percent
FROM score_table;
```

D 其他选项均不符合预期

正确答案：B　你的答案：C

官方解析：
选B，MySQL中不存在quantile_rank、percentile_rank函数，AC选项执行时会报错。
PERCENT_RANK()函数返回一个从0到1的数字，计算方法：按照partition by在分区内根据order by排序，结果=（相对位置-1）/（总行数-1）
注意：对于重复值，计算的时候，取重复值的第一行的位置。

```
PERCENT_RANK()
    OVER (
        PARTITION BY expr,...
        ORDER BY expr [ASC|DESC],...
    )
```

A
```
mysql> SELECT stu_id , subject_name ,score ,
    -> concat(round( quantile_rank() over(partition by subject_name order by score),4)*100,'%') AS percent
    -> FROM score_table;
ERROR 1064 (42000): You have an error in your SQL syntax; check the manual that corresponds to your MySQL server
FROM scor' at line 2
```

B
```
mysql> SELECT stu_id , subject_name ,score ,
    -> concat(round( percent_rank() over(partition by subject_name order by score),4)*100,'%') AS percent
    -> FROM score_table;
------+--------------+-------+---------+
u_id | subject_name | score | percent |
```

5. Mysql中表student_table(id,name,birth,sex)，查询张姓、李姓的学生总人数，错误的是（）？

A
```
select sum(case when name like '张%' then 1 else 0 end) as zhang_first_name ,
sum(case when name like '李%' then 1 else 0 end) as li_first_name
from
student_table;
```

B
```
select count(case when name like '张%' then 2 else null end) as zhang_first_name ,
count(case when name like '李%' then 2 else null end) as li_first_name
from
student_table;
```

C
```
select count(case when name like '张%' then 1 else 0 end) as zhang_first_name ,
count(case when name like '李%' then 2 else 0 end) as li_first_name
from
student_table;
```

D
```
select sum(case when name like '张%' then 1 else null end) as zhang_first_name ,
sum(case when name like '李%' then 1 else null end) as li_first_name
from
student_table;
```

正确答案：C

官方解析：
A是求和，加0时和不变，正确；
B此时只对2出现的次数做累加，对null不做次数的累加，正确；
C会对1和0出现的次数均做累加，2个结果均是所有记录的总行数，结果错误；
D是求和，加null时和不变，正确。

知识点：数据库、SQL

这道题考察的是 MySQL 中 `COUNT()` 和 `SUM()` 函数在条件统计时的区别：

- **`SUM()`**：会对所有非 `NULL` 的值进行求和。如果 `CASE WHEN` 不匹配时返回 `0`，那么 `SUM` 会把这些 `0` 也加进去，导致结果偏大；如果返回 `NULL`，`SUM` 会忽略 `NULL`，结果正确。
- **`COUNT()`**：只会统计非 `NULL` 的值。如果 `CASE WHEN` 不匹配时返回 `0`，那么 `COUNT` 会把这些 `0` 也当作有效值进行统计，导致结果偏大；只有当不匹配时返回 `NULL`，`COUNT` 才会忽略，结果才正确。

选项分析 “then 2 else null”意味着：当条件满足时，返回2；否则，返回NULL。

- **A 选项**：使用 `SUM`，不匹配时返回 `0`。`SUM(1)` 会正确统计张姓人数，`SUM(0)` 对结果无影响，**正确**。
- **B 选项**：使用 `COUNT`，不匹配时返回 `NULL`。`COUNT(2)` 会正确统计张、李姓人数，**正确**。
- **C 选项**：使用 `COUNT`，不匹配时返回 `0`。`COUNT(1)` 和 `COUNT(2)` 都会把 `0` 当作有效值，导致统计的人数等于表中总人数，**错误**。
- **D 选项**：使用 `SUM`，不匹配时返回 `NULL`。`SUM(1)` 会正确统计张、李姓人数，**正确**。

所以，错误的选项是 **C** ✅